

Project Report Computer

Programming-I (MCT-142L)

## Bank Account Management System

---



Submitted by: 2024-MC-15 Muhammad Shahzaib Aarbi

Submitted to: Mr. M. Ahsan Naeem

---

Department of Mechatronics & Control Engineering University  
of Engineering & Technology, Lahore.

May 13, 2025

## Table of Contents

|                                            |    |
|--------------------------------------------|----|
| Abstract:.....                             | 3  |
| Introduction: .....                        | 3  |
| Key features:.....                         | 3  |
| Instructions: .....                        | 4  |
| Sample Output: .....                       | 6  |
| Methodology.....                           | 7  |
| Algorithm Overview .....                   | 9  |
| Flowchart: .....                           | 10 |
| Conclusions and Future Enhancements: ..... | 11 |
| Future modification .....                  | 11 |
| Reviews .....                              | 11 |
| Reference: .....                           | 12 |
| Appendix: .....                            | 12 |

## Abstract:

This is the semester project of course (Computer Programming -1, C language). The program is a simple Bank management system in which there are multiple users and only one admin. Users can access only their own account while admin can access and change any of the user's data. Admin username and password is in built and is unchangeable. Moreover, users have following facilities: Make transactions (Deposit, withdraw and transfer), View transaction history and create new account. The program majorly uses following concepts of C language: Functions, Pointers, arrays and strings, Structures, File handling and Console display.

## Introduction:

Welcome to "Bank Account" management system. This is a simple and easy to use bank management system. At bank we have different users and admin. Everyone has its own authority and limitations. So, this program puts both the admin and user to its place. Further, we will discuss the key features, instructions and sample outputs of program.

## Key features:

1. Account Management: Easily create, access, and update bank accounts.
2. Secure Login: Account holder and administrator login functionalities ensure secure access.
3. Transaction Handling: Perform various transactions including withdrawals, deposits, and funds transfers.
4. Transaction History: Access transaction history for each account to track financial activities.
5. Administrator Privileges: Admin login enables viewing user accounts and their details for management purposes.
6. User-Friendly Interface: Intuitive interface for seamless navigation and operation.

## Instructions:

After executing the program an interface appears giving four choices to proceed:

### 1. Administrator:

- Enter '1' to Login as admin
- Use the predefined username and password.
- Admin username is “admin” and password is,“admin123”.
- Upon successful login, select from the following actions:
- Press '1' to view user by index number.
- Press '2' to view user accounts by username.
- Press '3' to View all accounts.
- Press '4' to Return to main menu.

### 2. Account Holder:

- Enter '2' to login as an account holder.
- Use username and password.
- After this Account Holder menu will appear.
- Press '1' to make transactions (transfer, deposit and withdraw).
- Press '2' to view transaction history.
- Press '3' to return to main menu.
- Press '4' to exit.

### 3. Creating a New Account:

- Choose '3' from the main menu to create a new account.
- Enter your desired username, password, email, and contact details.
- Optionally, deposit funds into the newly created account.
- After account creation, choose to deposit funds or exit.

### 4. Transaction Instructions:

#### 1. Withdrawal:

- To withdraw funds, select '3' from the Transaction Menu.
- Enter the desired withdrawal amount.
- If the amount is valid, the withdrawal is processed.
- Upon successful deposit, the new balance is shown, and you're returned to the account menu.

#### 3. Funds Transfer:

- Opt for '1' from the transaction menu to transfer funds.
- Enter the recipient's username and the amount you want to transfer.
- If the amount is valid and sufficient, the transaction is completed, and both sender and recipient histories are updated.

### 5. Return to Account Menu:

- Select '5' to return to Account menu.

### Sample Output:

| Account holder login                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Create new account                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Admin Login                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div style="border: 1px solid black; padding: 10px; text-align: center;"> Welcome to Banking system </div> <p>1. Admin Login.<br/> 2. Account Holder Login.<br/> 3. Create new Account.<br/> 4. Exit.</p> <p>Enter your choice: 2</p> <p>Enter your username: Shahzaib</p> <p>Enter your password:<br/> Shah123</p> <p>Login successful!</p> <p>Username: Shahzaib</p> <p>Balance: 1500.00</p> <p>Email:<br/> shahzaibhussain819@gmail.com</p> <p>Contact: 03196336397</p> <p>Press '1' to make a transaction. Press<br/> '2' to see transaction history.<br/> Press '3' to Return to main menu.<br/> Press '4' to exit the program.</p> <p>Enter your Choice: 1</p> <p>1. Transfer Funds<br/> 2. Deposit Money<br/> 3. Withdraw Money<br/> 4. View Transaction History<br/> 5. Return to Account</p> <p>Enter your choice: 1</p> <p>Your current balance: 108000.00<br/> Enter recipient username: Dayyan<br/> Enter amount to transfer: 1000<br/> Your new balance is: 500</p> <p>Press any key to return....</p> | <div style="border: 1px solid black; padding: 10px; text-align: center;"> Welcome to Banking system </div> <p>1.Admin Login.<br/> 2.Account Holder Login.<br/> 3.Create new Account.<br/> 4.Exit.</p> <p>Enter your choice: 3</p> <p>Username: Dayyan</p> <p>Password: dayyan12</p> <p>Account created! Your number is : 2</p> <p>Would you like to make an initial<br/> deposit? (Y/N): Y</p> <p>Current balance: 0.00</p> <p>Enter amount to deposit: 100000</p> <p>New balance: 100000.00</p> <p>Press any key to return...</p> | <div style="border: 1px solid black; padding: 10px; text-align: center;"> Welcome to Banking system </div> <p>1.Admin Login.<br/> 2.Account Holder Login.<br/> 3.Create new Account.<br/> 4.Exit.</p> <p>Enter your choice: 1</p> <p>Enter admin Username:<br/> admin</p> <p>Enter admin Password: admin123</p> <p>Admin Login successful!</p> <p>1. View Account by Index<br/> 2. View Account by Username<br/> 3. View All Accounts<br/> 4. Return to Main Menu</p> <p>Enter your choice: 3</p> <p>0. Shahzaib (Balance: 107000.00)<br/> 1. Dayyan (Balance: 13700.00)<br/> 2. Atiq (Balance: 100000.00)<br/> 3. Usman (Balance: 100100.00)<br/> 4. Ramaz (Balance: 100000.00)<br/> 5. Ahmad (Balance: 10000.00)<br/> 6. Zain (Balance: 100000.00)<br/> 7. Adeel (Balance: 1000000.00)<br/> 8. sample (Balance: 100000.00)</p> <p>Press any key to return...</p> |

## Methodology

As my project was to make bank management system from the little information given in project. So first came the stage of Research. Before that day I didn't know much about bank management. I started with chatgpt. I got my 50-60% information from chatgpt.

Following are the techniques I used while writing my code:

- I decided the functions and started tackling these features one by one. I first created a function (account holder login) separately, checked its each and everything. Then I made a new file copied previous code and wrote new function checked it and so on. Using this technique, I was able to handle my program, it seem simple and wasn't creating mess.
- Gpt is a great thing of our time. It can create the whole program but has limitations. On the other hand it is a unique teacher. I frequently used gpt (Chatgpt, DeepSeek and CoPilot) to learn new things. I approached chatgpt for everything I stuck at. Mostly while debugging my code, chatgpt was very helpful. Our knowledge is very small so chatgpt give new point of view with its vast knowledge.
- While debugging my program I first read my code then run it if this didn't solve the problem than, I paste my code in chatgpt for help. Most of the time chatgpt solve the problem but at some stages gpts also stop working. I tried my best while debugging the code since then there are some issues in my code which are out of my knowledge and also I didn't find any help from gpts.

Below is a breakdown of the main components of code.

### 1. Structuring Data with BankAccount Struct:

- The BankAccount structure encapsulates the essential information for a bank account, including username, password, balance, email, and contact information.

### 2. Dynamic Memory Allocation:

- Dynamic memory allocation using `realloc` is employed to manage an array of `BankAccount` structures (info). This allows the program to adapt to varying numbers of accounts without a fixed limit.
3. File Handling for Data Persistence:
    - File handling with `fopen`, `fprintf`, and `fclose` is utilized to read and write account details to a text file (`accounts.txt`). This file serves as a data store, ensuring data persistence between program executions.
  4. User Interface:
    - The console-based user interface is implemented using `printf`, `scanf`, and functions for displaying messages and gathering user input.
  5. Authentication:
    - Authentication is implemented with username and password verification for both account holders and the admin. It ensures only authorized users can access their accounts or the admin interface.
  6. Transaction Processing:
    - Transaction functionalities such as deposit, withdrawal, and funds transfer are provided. Each transaction is logged in a separate text file for audit trail purposes.
  7. Admin Interface:
    - An admin login interface allows privileged access to view user account details. Admins can view account details either by providing the index number or use

## Algorithm Overview

### 1. Initialization:

- The program starts by initializing variables, loading existing account details from the accounts.txt file into memory, and allocating memory for transaction files.

### 2. Main Menu Loop:

- The program enters a loop to display the main menu options.
- Users can choose from options like admin login, account holder login, account creation, or exit.

### 3. Admin Login:

- The admin login process verifies the entered username and password against predefined credentials (admin and admin123).
- Upon successful login, the admin can choose to view user account details by index number or username.

### 4. Account Holder Login:

- Account holder login prompts for username and password.
- Upon successful authentication, users can view account details, perform transactions, or view transaction history.

### 5. Account Creation:

- New users can create accounts by providing username, password, email, and contact information.
- Account details are saved to the accounts.txt file, and the user can choose to deposit funds or view account details immediately after account creation.

### 6. Transaction Processing:

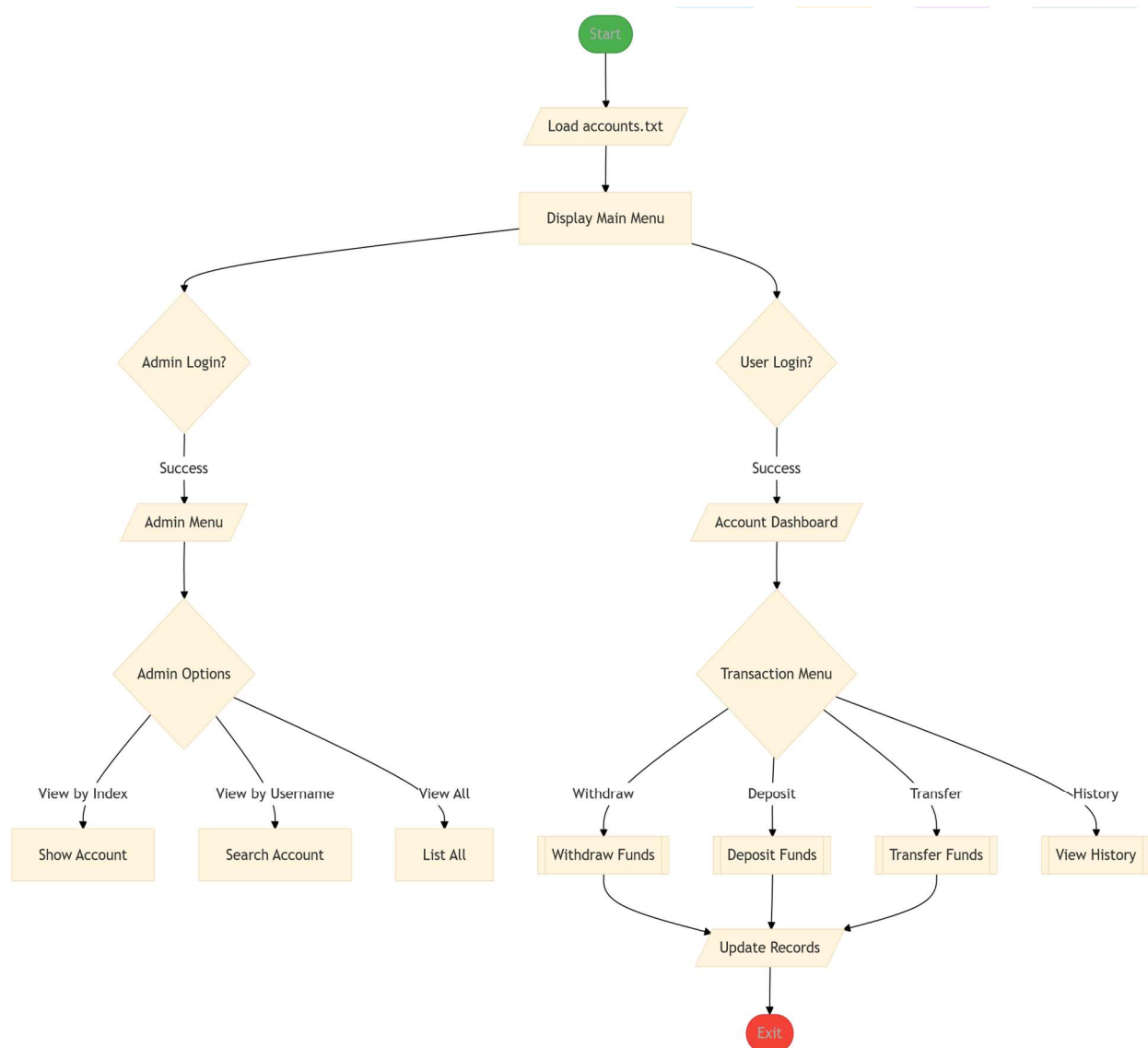
- Users can choose to deposit, withdraw, or transfer funds.
- Each transaction updates the account balance and logs the transaction details in the respective user's transaction file.

## 7. Data Persistence:

- Account details are updated in the accounts.txt file after every transaction or account creation.
- Transaction history is stored in separate text files for each user

### Flowchart:

Flow chart of program is given bellow:



## Conclusions and Future Enhancements:

This project was a comprehensive task for our learning throughout the semester. This task most frequently used our knowledge of pointers, functions and file handling. I faced some issues infrequently:

1. Dynamic memory allocation and then freeing up the memory after use.
2. Scanf is very odd function, it most often doesn't take input especially characters. A special care is needed while using it. In my code I don't know and found no logical reason for scanf not taking username string input in my accountholder login. Still this issue is unsolved.
3. I faced problems while using conditions in if else and loops.
4. In file handling while printing a string using fprintf function corrupts my accounts.txt file: it writes some data as it was supposed but at some places prints symbols at the place of characters.

## Future modification

There are many features that can be incorporated in this code to enhance its usability and user experience. Some of them are:

- User type: A simple user and a company client user.
- Account type: We can add user account type i.e., Bronze, silver, gold.
- Username: This code has single word username, it can be modified to name with spaces.
- Checks: More checks on password, email, name can be added to get valid information.
- Admin information: Can be modified to change the admin information during run time.

## Reviews

This course was more helpful in learning logic building rather than learning syntax of language. We covered a lot of content in small time and it was difficult to understand all these concept

properly. Those who don't have prior knowledge of programming lags despite of their capabilities as it is difficult to grasp the concepts of programming.

## Reference:

1. For flowchart generation I took help from DeepSeek.
2. Added Loading Animations using ChatGpt in the code.
3. Added Colors for error handling using the help of ChatGpt.

## Appendix:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <conio.h>
#include <windows.h>
#include <ctype.h>
#include <time.h>

#define RED    "\x1B[31m" // ChatGPT
#define GREEN  "\x1B[32m"
#define RESET "\x1B[0m"

// Bank account structure
typedef struct {
    char username[40];
    char password[9];
    float balance;
    char email[40];
    char contact[20];
} BankAccount;

// Global variables
BankAccount *accounts = NULL; // Dynamic array of all accounts
int account_count = 0; // Number of accounts
FILE **transaction_files = NULL; // Stores transaction history files

// Display functions
void displayHeader(const char *title);
void displayMenu();
void accountHolderLogin();
void adminLogin();
```

```

void adminMenu();
void createAccount();
void viewAccountDetails(int index);
void makeTransactions(int account_index);
void withdraw(int account_index);
void deposit(int account_index);
void transferFunds(int account_index);
void showTransactionHistory(int account_index);
void updateRecords();
void exitProgram();
void rotateDashAnimation(int durationMs, int frameDelayMs);

int main() {
    system("cls");
    system("COLOR 1F");

    FILE *file = fopen("accounts.txt", "r");
    if (!file) {
        printf("\n\n\t\t%sError: Cannot open accounts file.%s\n", RED,
RESET);
        return 1;
    }

    int capacity = 5;
    accounts = malloc(capacity * sizeof(BankAccount));

    while (fscanf(file, "%39s\t%8s\t%f\t%39s\t%19s",
        accounts[account_count].username,
        accounts[account_count].password,
        &accounts[account_count].balance,
        accounts[account_count].email,
        accounts[account_count].contact) == 5) {

        account_count++;

        if (account_count == capacity) {
            capacity *= 2;
            BankAccount *temp = realloc(accounts, capacity *
sizeof(BankAccount));
            if (!temp) {
                printf("\n\n\t\t%sError: Memory full!%s\n", RED, RESET);
                free(accounts);
                fclose(file);
                return 1;
            }
        }
    }
}

```



```

// Display the main menu options
void displayMenu(){
    printf("\n\t\t1. Admin Login");
    printf("\n\t\t2. Account Holder Login");
    printf("\n\t\t3. Create new Account");
    printf("\n\t\t4. Exit");
}
// Update account details in the file
void updateRecords(){
    FILE *file = fopen("accounts.txt", "w");
    if (file == NULL)
    {
        printf("\n\t\t%sError: Could not update records file.%s\n", RED,
RESET);
        return;
    }

    for (int i = 0; i < account_count; i++)
    {
        fprintf(file, "%-39s\t%-8s\t%.2f\t%-39s\t%-19s\n",
                accounts[i].username,
                accounts[i].password,
                accounts[i].balance,
                accounts[i].email,
                accounts[i].contact);
    }

    fclose(file);
}
// Clean up and exit the program
void exitProgram(){
    updateRecords();
    free(accounts);
    free(transaction_files);
    printf("\n\t\tSaving data and exiting, please wait...");
    rotateDashAnimation(2000, 200);
    exit(0);
}
// Account holder login function
void accountHolderLogin() {
    char username[40], password[9];
    int attempts = 0;
    system("cls");
    system("COLOR 30");
    while (attempts < 3) {

```

```

displayHeader("Account Login");

printf("\n\t\tUsername: ");
scanf("%39s", username);

printf("\t\tPassword: ");
scanf("%8s", password);

printf("\n\t\tValidating credentials, please wait...");
rotateDashAnimation(1500, 200);
for (int i = 0; i < account_count; i++) {
    if (strcmp(username, accounts[i].username) == 0 &&
        strcmp(password, accounts[i].password) == 0) {
        printf("\n\t\t%sLogin successful!%s", GREEN, RESET);
        Sleep(1000);
        system("cls");
        viewAccountDetails(i);
        return;
    }
}
printf("\n\t\t%sInvalid login! Attempts left: %d %s", RED, 2 -
attempts, RESET);
Sleep(1500);
system("cls");
attempts++;
}

printf("\n\t\t%sToo many failed attempts. Returning to menu...%s", RED,
RESET);
rotateDashAnimation(1500, 200);
system("cls");
}
// Withdraw money from an account
void withdraw(int account_index) {
    system("cls");
    system("COLOR 70");
    float amount;
    displayHeader("Withdraw Money");
    printf("\n\t\tBalance: Rs. %.2f", accounts[account_index].balance);

    printf("\n\t\tEnter amount: ");
    scanf("%f", &amount);

    printf("\n\t\tProcessing your withdrawal, please wait...");
    rotateDashAnimation(2000, 200);

```

```

    if (amount <= 0) {
        printf("\n\t\t%sAmount must be positive!%s", RED, RESET);
    }
    else if (amount > accounts[account_index].balance) {
        printf("\n\t\t%sInsufficient funds!%s", RED, RESET);
    }
    else {
        accounts[account_index].balance -= amount;
        updateRecords();

        // Get the current date and time
        time_t t;
        struct tm *tm_info;
        time(&t);
        tm_info = localtime(&t);
        char date_time[30];
        strftime(date_time, sizeof(date_time), "%Y-%m-%d %H:%M:%S",
tm_info);

        char filename[20];
        sprintf(filename, "User%d.txt", account_index + 1);
        FILE *file = fopen(filename, "a");
        fprintf(file, "Withdrawal: Rs. %.2f on %s\n", amount, date_time);
        fclose(file);

        printf("\n\t\t%sSuccess! New balance: Rs. %.2f%s", GREEN,
accounts[account_index].balance, RESET);
    }

    printf("\n\t\tPress any key to continue...");
    getch();
    system("cls");
}
// Deposit money into an account
void deposit(int account_index){
    system("COLOR 70");
    float amount;

    displayHeader("Deposit Money");
    printf("\n\t\tCurrent balance: %.2f", accounts[account_index].balance);
    printf("\n\t\tEnter amount to deposit: ");
    scanf("%f", &amount);

    printf("\n\t\tProcessing your withdrawal, please wait...");
    rotateDashAnimation(2000, 200);

```

```

while (amount <= 0)
{
    printf("\n\t\t%sAmount must be positive! Try again: %s", RED,
RESET);
    scanf("%f", &amount);
}

accounts[account_index].balance += amount;

// Get the current date and time
time_t t;
struct tm *tm_info;
time(&t);
tm_info = localtime(&t);
char date_time[30];
strftime(date_time, sizeof(date_time), "%Y-%m-%d %H:%M:%S", tm_info);

char filename[20];
sprintf(filename, "User%d.txt", account_index + 1);
FILE *file = fopen(filename, "a");
fprintf(file, "Deposit: Rs. %.2f on %s\n", amount, date_time);
fclose(file);

printf("\n\t\t%sDeposit successful!%s", GREEN, RESET);
printf("\n\t\tNew balance: %.2f", accounts[account_index].balance);
updateRecords();

printf("\n\t\tPress any key to return...");
getch();
system("cls");
}
// Transfer funds between accounts
void transferFunds(int sender_index){
    system("COLOR 70");
    char recipient[40];
    float amount;
    int recipient_index = -1;

    displayHeader("Transfer Funds");
    printf("\n\t\tYour current balance: %.2f",
accounts[sender_index].balance);

    printf("\n\t\tEnter recipient username: ");
    scanf("%39s", recipient);

```

```

for (int i = 0; i < account_count; i++)
{
    if (strcmp(recipient, accounts[i].username) == 0)
    {
        recipient_index = i;
        break;
    }
}

if (recipient_index == -1)
{
    printf("\n\t\t%sRecipient not found!%s", RED, RESET);
    printf("\n\t\tPress any key to return...");
    getch();
    system("cls");
    return;
}

printf("\n\t\tEnter amount to transfer: ");
scanf("%f", &amount);
printf("\n\t\tProcessing your withdrawal, please wait...");
rotateDashAnimation(2000, 200);
if (amount <= 0)
{
    printf("\n\t\t%sAmount must be positive!%s", RED, RESET);
}
else if (amount > accounts[sender_index].balance)
{
    printf("\n\t\t%sInsufficient funds!%s", RED, RESET);
}
else
{
    accounts[sender_index].balance -= amount;
    accounts[recipient_index].balance += amount;

    // Get the current date and time
    time_t t;
    struct tm *tm_info;
    time(&t);
    tm_info = localtime(&t);
    char date_time[30];
    strftime(date_time, sizeof(date_time), "%Y-%m-%d %H:%M:%S",
tm_info);

    char filename[20];

```

```

        sprintf(filename, "User%d.txt", sender_index + 1);
        FILE *file = fopen(filename, "a");
        fprintf(file, "Transfer to %s: Rs. %.2f on %s\n", recipient,
amount, date_time);
        fclose(file);

        sprintf(filename, "User%d.txt", recipient_index + 1);
        file = fopen(filename, "a");
        fprintf(file, "Transfer from %s: Rs. %.2f on %s\n",
accounts[sender_index].username, amount, date_time);
        fclose(file);

        printf("\n\t\t%sTransfer successful!%s", GREEN, RESET);
        printf("\n\t\tYour new balance: %.2f",
accounts[sender_index].balance);
        updateRecords();
    }

    printf("\n\t\tPress any key to return...");
    getch();
    system("cls");
}
// Show transaction history for an account
void showTransactionHistory(int account_index){
    system("COLOR 70");
    char filename[20];
    sprintf(filename, "User%d.txt", account_index + 1);

    displayHeader("Transaction History");
    printf("\n\t\tLoading transaction history, please wait...");
    rotateDashAnimation(1500, 200);
    FILE *file = fopen(filename, "r");
    if (file == NULL)
    {
        printf("\n\t\t%sNo transaction history found.%s", RED, RESET);
    }
    else
    {
        char line[100];
        while (fgets(line, sizeof(line), file))
        {
            printf("\n\t\t=====");
            printf("\n\t\t%s", line);
        }
    }
}

```

```

        fclose(file);
    }

    printf("\n\t\tPress any key to return...");
    getch();
    system("cls");
}
// Transaction menu for an account
void makeTransactions(int account_index){
    system("COLOR 70");
    int choice;
    do
    {
        displayHeader("Transaction Menu");
        printf("\n\t\t1. Transfer Funds");
        printf("\n\t\t2. Deposit Money");
        printf("\n\t\t3. Withdraw Money");
        printf("\n\t\t4. View Transaction History");
        printf("\n\t\t5. Return to Account");
        printf("\n\t\tEnter your choice: ");
        scanf("%d", &choice);
        system("cls");
        switch (choice)
        {
            case 1:
                printf("\n\t\tProcessing your request, please wait...");
                rotateDashAnimation(1500, 200);
                system("cls");
                transferFunds(account_index);
                break;
            case 2:
                printf("\n\t\tProcessing your request, please wait...");
                rotateDashAnimation(1500, 200);
                system("cls");
                deposit(account_index);
                break;
            case 3:
                printf("\n\t\tProcessing your request, please wait...");
                rotateDashAnimation(1500, 200);
                withdraw(account_index);
                break;
            case 4:
                printf("\n\t\tLoading transaction history, please wait...");
                rotateDashAnimation(1500, 200);
                system("cls");

```

```

        showTransactionHistory(account_index);
        break;
    case 5:
        return;
    default:
        printf("\n\t\t%sInvalid choice! Try again.%s", RED, RESET);
        Sleep(1000);
        system("cls");
    }
} while (1);
}
// View account details and options
void viewAccountDetails(int account_index){
    system("COLOR 70");

    printf("\n\t\tLoading account details, please wait...");
    rotateDashAnimation(1500, 200);
    system("cls");

    int choice;
    do
    {
        displayHeader("Account Details");
        printf("\n\t\tUsername: %s", accounts[account_index].username);
        printf("\n\t\tBalance: %.2f", accounts[account_index].balance);
        printf("\n\t\tEmail: %s", accounts[account_index].email);
        printf("\n\t\tContact: %s", accounts[account_index].contact);

        printf("\n\n\t\t1. Make Transactions");
        printf("\n\t\t2. View Transaction History");
        printf("\n\t\t3. Return to Main Menu");
        printf("\n\t\t4. Exit");
        printf("\n\t\tEnter your choice: ");
        scanf("%d", &choice);

        system("cls");

        switch (choice)
        {
            case 1:
                makeTransactions(account_index);
                break;
            case 2:
                showTransactionHistory(account_index);
                break;

```

```

        case 3:
            return;
        case 4:
            exitProgram();
        default:
            printf("\n\t\t%sInvalid choice! Try again.%s", RED, RESET);
            Sleep(1000);
            system("cls");
    }
} while (1);
}
// Create a new bank account
void createAccount() {
    system("COLOR 70");
    BankAccount new_acc;
    system("cls");
    displayHeader("Create New Account");

    printf("\n\t\tUsername: ");
    scanf("%39s", new_acc.username);

    printf("\t\tPassword (max 8 chars): ");
    scanf("%8s", new_acc.password);

    printf("\t\tEmail: ");
    scanf("%39s", new_acc.email);

    printf("\t\tContact: ");
    scanf("%19s", new_acc.contact);

    new_acc.balance = 0.0;

    account_count++;
    accounts = realloc(accounts, account_count * sizeof(BankAccount));
    accounts[account_count - 1] = new_acc;

    updateRecords();

    char filename[20];
    sprintf(filename, "User%d.txt", account_count);
    FILE *file = fopen(filename, "w");
    if (file) fclose(file);
    printf("\n\t\tCreating your account, please wait...");
    rotateDashAnimation(2000, 200);
}

```

```

        printf("\n\t\t%sAccount created! Your Number is : %d %s", GREEN,
account_count, RESET);

    char choice;
    printf("\n\t\tWould you like to make an initial deposit? (Y/N): ");
    scanf(" %c", &choice);

    if (toupper(choice) == 'Y') {
        system("cls");
        deposit(account_count - 1);
    } else {
        printf("\n\t\tPress any key to return to main menu...");
        getch();
        system("cls");
    }
}

// Admin login and functions
void adminLogin(){
    const char admin_username[] = "admin";
    const char admin_password[] = "admin123";
    char username[40], password[40];
    int attempts = 0;

    while (attempts < 3)
    {
        system("cls");
        system("COLOR 30");
        displayHeader("Admin Login");

        printf("\n\t\tEnter admin username: ");
        scanf("%39s", username);

        printf("\t\tEnter admin password: ");
        scanf("%39s", password);

        printf("\n\t\tValidating credentials, please wait...");
        rotateDashAnimation(1500, 200);
        if (strcmp(username, admin_username) == 0 &&
            strcmp(password, admin_password) == 0)
        {
            printf("\n\t\t%sAdmin login successful!%s", GREEN, RESET);
            Sleep(1000);
            system("cls");
            adminMenu();
            return;
        }
    }
}

```

```

    }
    printf("\n\t\t%sInvalid credentials! Attempts remaining: %d
%s", RED, 2 - attempts, RESET);
    Sleep(1500);
    system("cls");
    attempts++;
}

printf("\n\t\t%sToo many failed attempts. RetAurning to main menu.%s",
RED, RESET);
rotateDashAnimation(1500, 200); // 1.5 seconds animation
system("cls");
}
// Admin menu and functions
void adminMenu(){
    system("COLOR 08");
    int choice;
    do
    {
        printf("\n\t\tLoading Admin Menu...");
        rotateDashAnimation(1000, 150);
        system("cls");
        displayHeader("Admin Menu");
        printf("\n\t\t1. View Account by Index");
        printf("\n\t\t2. View Account by Username");
        printf("\n\t\t3. View All Accounts");
        printf("\n\t\t4. Return to Main Menu");
        printf("\n\t\tEnter your choice: ");
        scanf("%d", &choice);

        system("cls");

        switch (choice)
        {
            case 1:
            {
                int index;
                printf("\n\t\tEnter account index (0-%d): ", account_count -
1);

                scanf("%d", &index);

                if (index >= 0 && index < account_count)
                {
                    viewAccountDetails(index);
                }
            }
        }
    }
    while (choice != 4);
}

```

```

        else
        {
            printf("\n\t\t%sInvalid index!%s", RED, RESET);
            Sleep(1000);
        }
        break;
    }
    case 2:
    {
        char username[40];
        printf("\n\t\tEnter username: ");
        scanf("%39s", username);

        int found = 0;
        for (int i = 0; i < account_count; i++)
        {
            if (strcmp(username, accounts[i].username) == 0)
            {
                viewAccountDetails(i);
                found = 1;
                break;
            }
        }

        if (!found)
        {
            printf("\n\t\t%sAccount not found!%s", RED, RESET);
            Sleep(1000);
        }
        break;
    }
    case 3:
        printf("\n\t\tLoading all accounts, please wait...");
        rotateDashAnimation(1500, 200);
        system("cls");
        displayHeader("All Accounts");
        for (int i = 0; i < account_count; i++)
        {
            printf("\n\t\t%d. %s (Balance: %.2f)", i,
accounts[i].username, accounts[i].balance);
        }
        printf("\n\t\tPress any key to return...");
        getch();
        system("cls");
        break;

```

```

        case 4:
            return;
        default:
            printf("\n\t\t%sInvalid choice!%s", RED, RESET);
            Sleep(1000);
            system("cls");
    }
} while (1);
}

void rotateDashAnimation(int durationMs, int frameDelayMs) {
    char dashPositions[] = {'-', '\\', '|', '/'};
    int frames = durationMs / frameDelayMs;

    for (int i = 0; i < frames; i++) {
        printf("\b%c", dashPositions[i % 4]);
        fflush(stdout);
        Sleep(frameDelayMs);
    }
    printf("\b \b");
}

```